THE BIOROBOTICS INSTITUTE

Scuola Superiore Sant'Anna

Introduction to ROS Programming with Python

Egidio Falotico
Ugo Albanese

{e.falotico, u.albanese}@santannapisa.it

Final Project: Dustbot

DEADLINE

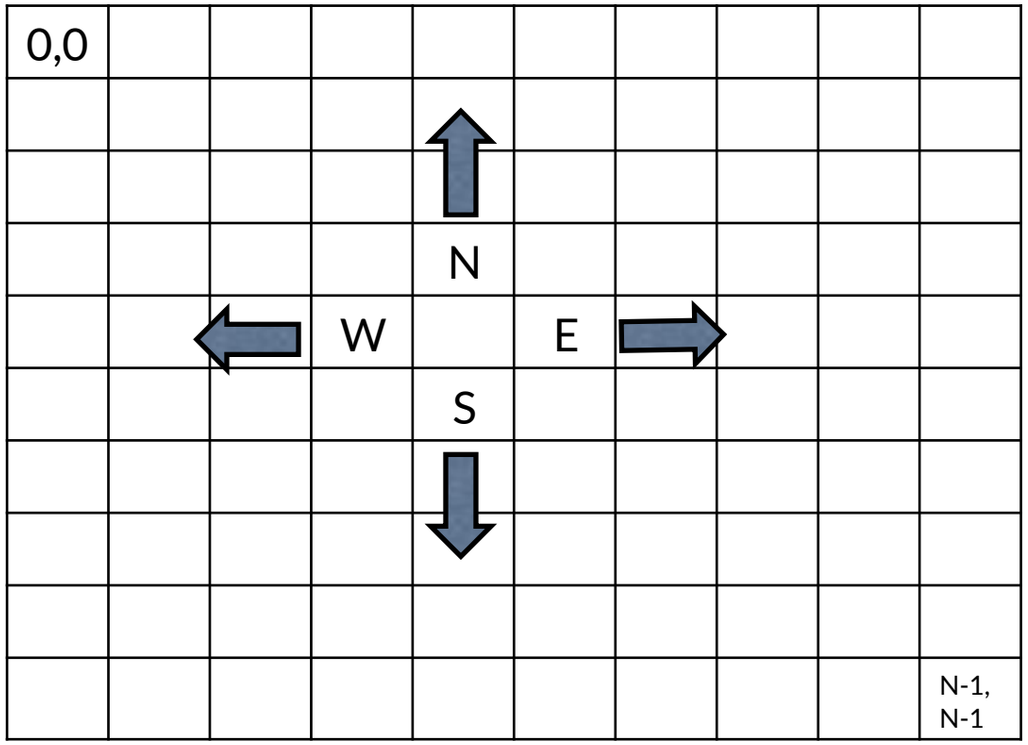
??/12/25

Final Project: Dustbot

- *Dustbot 2.0*, a mobile robot, navigates a 2-dimensional *NxN* grid along the four directions (*N-S-E-W*) with a speed of ***one grid-cell/s***.

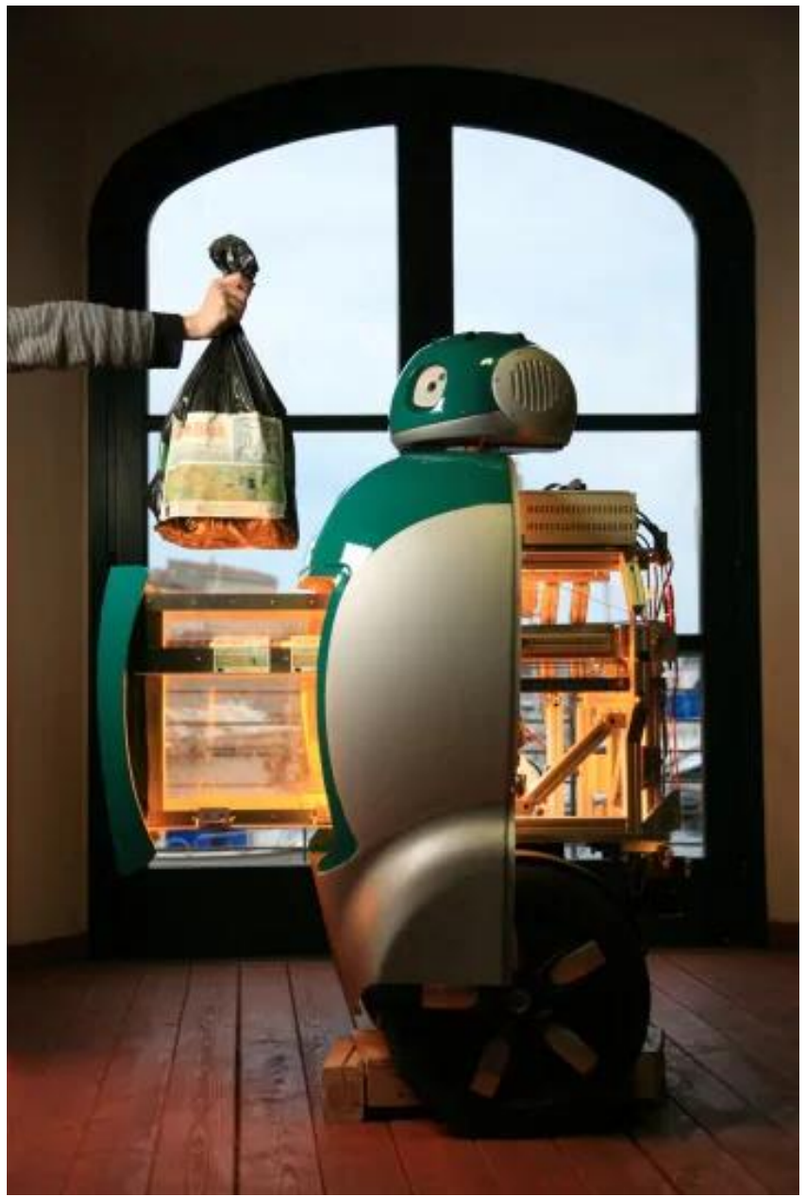
- Its global position in the grid in X,Y coordinates are published on the `/dustbot/global_position` ROS Topic.

- The direction in which the robot moves can be ***altered*** using the custom ROS service `/dustbot/set_direction`.



Final Project: Dustbot

- **Dustbot** needs to pick up garbage at specific world coordinates which, are available on the `/dustbot/garbage_position` topic.
- **Monitoring** it, *Dustbot* can continously figure out which **direction** to **move** towards.
- Once its destination is reached, *Dustbot* has to initiate the **garbage pick up** calling the `/dustbot/load_garbage` service (picking up will succed **only** at garbage_position, *instantaneously for simplicity*)
- Once the garbage has been loaded, a new random garbage position gets published for **Dustbot** to collect.
- After **P** pick-ups the simulation ends.



Final Project: Dustbot
Some Tips

- **Implement** 2 ROS nodes in 2 Python scripts
  - A **world node** (publishers and service server)
    - Choose/create suitable messages
    - Implement custom services (definitions and servers)
    - Moves the "simulated robot" regardless of the "*control*" node (i.e. robot)
  - A **robot node** (subscribers and service client)
    - Computes the direction to take and sets it by calling `/dustbot/set_direction`
- The Size of the grid (**`N`**) and number of pick-ups (**`P`**) should be **loaded on startup** as parameters
- Use a `launch` file to run the nodes and load the `N` and `P` parameters

Final Project: Dustbot

CAUTION:

- Create a separate `dustbot_interface` package for the message/service definitions
- Don't forget to add any new package dependency in `CMakeList.txt` and `package.xml`, e.g. the package of any message that you'd use.
- Re-build the package after any change to code and configuration files
- For the service client use `Client.call_async()` and add to the returned future object a callback that will check the response of service and act accordingly (`future.add_done_callback(clbk_fun)`)

```
...

# local nested function
def _set_direction_done_clbk(future):
    # here the future is done and response is available
    response = future.result()
    # inspect the response and act accordingly

future = self.set_direction_client.call_async(set_dir_req)
future.add_done_callback(_set_direction_done_clbk)

...
```

Final Project: Dustbot

HOW TO SUBMIT THE CODE

- Send a zip archive containing ONLY the packages
  - `dustbot`
  - `dustbot_interface`
- In the `dustbot` package directory, add a `Readme.md` file briefly detailing the design choices/assumptions about the code and how to run the application
  - TO: *u.albanese@santannapisa.it*
  - CC: *e.falotico@santannapisa.it*